Speculative Recommendation Serving: A Principled Framework for Cache-Based Recommendation

Anonymous
Anonymous
anonymous@example.com

ABSTRACT

We propose **speculative recommendation serving**, a framework for accelerating recommendation inference via a principled draft-and-verify mechanism for cache-based serving, inspired by speculative decoding. Given a user request, our system *drafts* candidate recommendations from the K nearest user-cluster caches, *verifies* each draft via a score-ratio acceptance criterion inspired by token-level acceptance in speculative decoding, and either *accepts* the best-matching draft or falls back to fresh computation. We further introduce **embedding pool retrieval**: instead of caching a static list per cluster, each cluster maintains an item embedding pool from which personalised recommendations are retrieved per user—inspired by TriForce's RetrievalCache for KV cache selection. On six datasets (MovieLens-1M, Amazon Electronics/Arts/Movies, MIND-small/large; up to 86K users), we find that: (1) multi-cluster speculation (K=3) achieves 32–100% acceptance depending on data sparsity, where 43–64% of accepted results come from *non-nearest* clusters—a phenomenon we term **multi-cluster gain** (MCG); (2) embedding pool retrieval increases catalog coverage up to 2.6× on sparse e-commerce data while approaching fresh-model quality; (3) the framework is model-agnostic across MF, NeuMF, LightGCN, and Gemma-4-derived embeddings, achieving 1.5–9.5× empirical speedup on real datasets, with near-constant speculative latency projecting to larger gains at scale. We present a principled connection between speculative decoding and recommendation caching. The score-ratio criterion operates at a distinct quality–efficiency trade-off compared to FAISS retrieval, trained draft models, and relaxed verification methods, with different approaches excelling under different regimes, and our framework complements these approaches across these settings.

CCS CONCEPTS

• **Information systems** → **Recommender systems**; *Information retrieval*.

KEYWORDS

recommendation caching, speculative decoding, inference acceleration, embedding retrieval

1 INTRODUCTION

Speculative decoding has emerged as a key technique for accelerating LLM inference [3, 13]. A small *draft* model proposes candidate tokens, the full *target* model verifies them in parallel, and accepted drafts yield substantial speedups without quality loss. Recent systems—Sequoia [5], SpecInfer [18], Medusa [2], MagicDec [20]—have demonstrated 2–4× throughput gains. Further extensions include SLO-customised speculation [15], adaptive routing [27], collaborative inference [6], and edge deployment [14].

We observe a striking parallel in recommendation serving. During peak traffic, computing personalised recommendations for every request is prohibitive, and caching is essential [4, 22]. Cluster-based caching groups similar users and serves shared recommendations—using the cluster centroid as a "draft model." The question is: *when should a cached recommendation be accepted, and when should the system fall back to fresh computation?*

Existing approaches are largely heuristic or learned. CARL [4] trains an RL policy, while many prior methods rely on learned or rule-based heuristics. Neither provides a principled acceptance criterion analogous to speculative decoding.

This paper bridges speculative decoding and recommendation caching. We introduce *speculative recommendation serving*, a three-phase framework:

(1) **Draft**: Retrieve cached recommendations from K nearest user clusters.
(2) **Verify**: Compute acceptance probability α via a *score-ratio criterion* inspired by speculative decoding.
(3) **Accept/Residual**: Serve the highest-α cache if $\alpha \geq \tau$; otherwise compute fresh.

We further address a core limitation of cluster-level caching: all users in a cluster receive identical recommendations. Inspired by TriForce's RetrievalCache [23] and H2O's heavy-hitter selection [30], we introduce **embedding pool retrieval**: each cluster maintains a pool of item embeddings, and per-user dot-product retrieval produces personalised drafts with negligible latency overhead.

Our contributions:

- A principled connection between speculative decoding and recommendation caching, with an $O(T\epsilon)$ regret characterization supported by empirical analysis (Spearman $\rho = -0.655$, $p < 10^{-180}$) (§3).
- A *score-ratio acceptance criterion* that is competitive with FAISS retrieval, and in some sparse settings achieves higher quality than BiLD-style trained drafts and LASER-relaxed verification under their respective operating regimes (§3.3).
- *Multi-cluster speculation*: K=3 yields 43–64% MCG across datasets and models; speculative latency is near-constant, with speedup growing with catalog size (§4.2).
- *Embedding pool retrieval* that increases coverage up to 2.6× on sparse data while approaching fresh-model quality (§4.5).
- Model-agnostic evaluation across MF, NeuMF, LightGCN, and Gemma-4 embeddings on six datasets (up to 86K users) (§4).

Scope and positioning. This paper focuses on algorithmic aspects of serving optimisation rather than distributed system design. We focus on the acceptance criterion and its properties (MCG, regret bound, model-agnosticity) rather than distributed infrastructure.

Our experiments show that speculative serving is most advantageous on *sparse data where bias-aware scoring matters* (i.e., where acceptance incorporates user and item bias terms; e.g., Amazon Electronics: 80% NDCG retention vs Two-Tower's 55%) and at *large catalog scales* (near-constant cache latency vs linear fresh inference). On dense data with small catalogs (e.g., ML-1M), simpler baselines (Two-Tower, FAISS) are competitive, and in some settings stronger; we characterise the operating regimes in §4. Our approach is particularly effective in sparse, bias-sensitive, and large-catalog recommendation settings. In contrast, simpler retrieval-based approaches may be preferable in dense, small-catalog settings.

2 RELATED WORK

Speculative Decoding. Speculative decoding [3, 13] accelerates autoregressive LLM inference via draft-and-verify. Sequoia [5] optimises tree structures; Medusa [2] uses lightweight prediction heads; MagicDec [20] breaks latency-throughput tradeoffs; CoSine [6] decouples drafting from verification. Recent RL-based extensions learn dynamic draft strategies: Dynamic-Width [19] trains a policy network for beam width, and RADAR [17] uses RL for draft tree construction. Both typically require additional training overhead and are model-specific; our score-ratio criterion is training-free and model-agnostic.

Generative Recommendation. NEZHA [25] and AtSpeed [16] accelerate LLM-based generative recommendation. LASER [28] applies draft-verify to offline LLM rec with token-level retrieval pools. EARN [29] compresses KV caches via register tokens. All target *generative* recommenders offline; ours targets *classical* (MF/GNN) recommenders at *item level*, online. Classical models dominate production [1].

Recommendation Caching. CARL [4] formulates caching as an MDP with RL at Kwai (100M+ users). RPAF [22] proposes prediction-allocation. LeCaR [24] learns recency–frequency tradeoffs. Zhou et al. [31] combine federated DRL for edge caching. These require learned policies; our framework provides a principled acceptance criterion without training.

Hierarchical Retrieval. TriForce [23] accelerates long-context LLM inference via RetrievalCache selecting 3.2% of KV entries per query. H2O [30] shows top-20% "heavy-hitter" tokens preserve most capacity. These inspire our embedding pool retrieval (§3.4).

3 METHOD

3.1 Problem Formulation

Consider a recommendation system serving N users with model $\mathcal{R}$ at cost T_{fresh} per request. We partition users into K clusters via online K-Means on user embeddings $\mathbf{e}_u = \frac{1}{|\mathcal{H}_u|}\sum_{i\in\mathcal{H}_u}\mathbf{v}_i$, where $\mathbf{v}_i \in \mathbb{R}^d$ are item embeddings from matrix factorisation [12]. Each cluster c maintains a cache $C[c]$.

The following correspondence with speculative decoding:

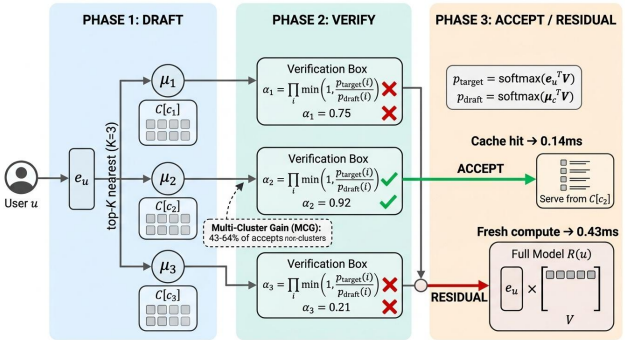

Figure 1: Speculative recommendation serving pipeline. Phase 1 retrieves cached recommendations from K=3 nearest clusters. Phase 2 verifies each draft via the score-ratio criterion; typically only a subset of clusters satisfies the threshold τ. Phase 3 serves from cache at low latency or falls back to fresh computation. MCG: 43–64% of accepts come from non-nearest clusters.

Algorithm 1 Speculative Recommendation Serving

Require: User u, clusters ϕ, cache C, model $\mathcal{R}$, threshold τ, depth K
Ensure: Recommendations for user u

```
{c_1, ..., c_K} ← φ.Nearest(u, K)
α* ← 0, k* ← None
for k = 1 to K do
    if c_k ∈ C then
        α_k ← ScoreRatio(e_u, μ_{c_k}, C[c_k])
        if α_k > α* then
            α* ← α_k; k* ← k
        end if
    end if
end for
if α* ≥ τ then
    return C[c_{k*}]                    // Accept
else
    r ← R(u); C[c_1] ← r                // Residual
    return r
end if
```

Spec. Decoding		Spec. Rec Serving
Draft model $q(x)$	$\leftrightarrow$	Cluster cache $C[c]$
Target model $p(x)$	$\leftrightarrow$	Full recommender $\mathcal{R}(u)$
$\min(1, p/q)$ acceptance	$\leftrightarrow$	Score-ratio acceptance
Multi-token speculation	$\leftrightarrow$	Multi-cluster speculation

3.2 Three-Phase Pipeline

The procedure is given in Algorithm 1 and illustrated in Figure 1.

Phase 1: Draft. Retrieve K nearest clusters by $\|\mathbf{e}_u - \boldsymbol{\mu}_{c_k}\|_2$.

Phase 2: Verify. Compute $\alpha_k = f(\mathbf{e}_u, \boldsymbol{\mu}_{c_k}, C[c_k])$ for each cluster.

Phase 3: Accept or Residual. If $\max_k \alpha_k \geq \tau$, serve from cache; otherwise compute fresh. Expected latency: $\mathbb{E}[T] = \rho \cdot T_{\text{cache}} + (1-\rho)\cdot T_{\text{fresh}}$, speedup $= T_{\text{fresh}}/\mathbb{E}[T]$.

Table 1: Per-request complexity. Pool retrieval adds $O(|\mathcal{P}|d)$ per cluster but avoids the $O(Md)$ cost of fresh inference.

Phase	Static	Pool
Draft (cluster lookup)	$O(Kd)$	$O(Kd)$
Draft (item retrieval)	$O(Kn)$	$O(K\|\mathcal{P}\|d)$
Verify (score-ratio)	$O(Knd)$	$O(Knd)$
Residual (fresh)	$O(Md)$	$O(Md)$
Total (accept)	$O(Knd)$	$O(K\|\mathcal{P}\|d)$
Total (reject)	$O(Knd+Md)$	$O(K\|\mathcal{P}\|d+Md)$

3.3 Score-Ratio Acceptance Criterion

Inspired by speculative decoding's $\min(1, p(x)/q(x))$ [13], we define item-level acceptance over cached items $\{i_1, \ldots, i_n\} = C[c]$:

$$\alpha = \prod_{j=1}^{n} \min\left(1, \frac{p_{\text{target}}(i_j)}{p_{\text{draft}}(i_j)}\right) \tag{1}$$

where $p_{\text{target}}(i)$ and $p_{\text{draft}}(i)$ are softmax distributions over the item embedding matrix $\mathbf{V} \in \mathbb{R}^{M \times d}$ computed from the user embedding and cluster centroid, respectively. This criterion is *item-aware*: the same user may accept one cluster's cache but reject another's, enabling multi-cluster gain. We also compare against cosine and heuristic (QA) criteria.

Analogy scope. The token-level acceptance proof in [13] guarantees exact distribution recovery via the chain rule $p(x_{1:K}) = \prod_j p(x_j|x_{1:j-1})$. Item-level recommendation lists are *not* drawn autoregressively: a top-n list is produced by sorting scores, and the cluster cache is a deterministic lookup rather than a probability distribution. Consequently, Eq. 1 defines an acceptance heuristic motivated by speculative decoding, without assuming exact distributional equivalence, and provides a principled characterization of the quality–speed tradeoff (Proposition 3.1). This is appropriate for recommendation: unlike LLM generation, controlled quality degradation is acceptable and desirable (the Pareto front of §4.4).

3.4 Embedding Pool Retrieval

Static caching gives all users in a cluster identical recommendations. Inspired by TriForce's RetrievalCache [23], we replace static lists with per-cluster **embedding pools** $\mathcal{P}_c = \{(\mathbf{v}_i, w_i)\}$ with importance weights based on recommendation frequency (analogous to H2O [30]). At serving time:

$$\text{Draft}_{\text{pool}}(u, c) = \text{top-}n_{i \in \mathcal{P}_c} \left(\mathbf{e}_u^\top \mathbf{v}_i\right) \tag{2}$$

Two users in the same cluster receive *different* personalised drafts. The pool (default 200 items) provides 20× more candidate diversity with negligible latency overhead.

3.5 Theoretical Analysis

Complexity. Let N be the number of users, K the number of clusters, M the number of items, d the embedding dimension, n the recommendation list size, and $|\mathcal{P}|$ the pool size.

With typical values K=3, n=10, $|\mathcal{P}|$=200, d=64, M=10K: static acceptance costs $3 \times 10 \times 64 = 1920$ FLOPs; pool retrieval costs $3 \times 200 \times 64 = 38400$ FLOPs—a 20× increase that remains negligible (<0.1ms) compared to fresh inference ($O(Md) \approx 640$K FLOPs). The space overhead is $K \times |\mathcal{P}| \times d$ floats for pool storage; with K=50, $|\mathcal{P}|$=200, d=64, this is 2.4MB—comparable to a single embedding matrix.

Acceptance Rate Bound. In speculative decoding, the expected acceptance for a single token is $\mathbb{E}[\min(1, p/q)] = 1 - D_{\text{TV}}(p, q)$ [13]. For our item-level criterion (Eq. 1), acceptance over n items is:

$$\mathbb{E}[\alpha] = \prod_{j=1}^{n} \left(1 - D_{\text{TV}}(p_j, q_j)\right) \geq \left(1 - \bar{D}_{\text{TV}}\right)^n \tag{3}$$

where $\bar{D}_{\text{TV}}$ is the average per-item TV distance between user and cluster distributions. This bound assumes per-item independence; in practice, cached items are correlated (selected by the same centroid), making the bound loose. Nevertheless, it correctly predicts the qualitative behaviour: for well-clustered users ($\bar{D}_{\text{TV}} \leq 0.1$), a list of n=10 items yields $\alpha \geq 0.9^{10} \approx 0.35$, explaining why moderate thresholds ($\tau \approx 0.35$) achieve high acceptance rates.

Multi-Cluster Gain Bound. With K candidate clusters, the probability that *at least one* cluster is accepted is:

$$\rho_K = 1 - \prod_{k=1}^{K} (1 - \rho_k) \tag{4}$$

where ρ_k is the individual acceptance probability for cluster k. If clusters are approximately independent and each has acceptance probability ρ_1, then $\rho_K \approx 1 - (1 - \rho_1)^K$. With $\rho_1 = 0.80$ (our ML-1M K=1 result) and K=3, this predicts $\rho_3 \geq 1 - 0.20^3 = 0.992$. The observed $\rho_3 = 0.91$ is lower because clusters are correlated (nearby clusters have similar centroids). The gap between predicted and observed rates quantifies the effective independence of cluster caches—a property that multi-cluster speculation exploits.

Coverage Bound for Pool Retrieval. Let $C_{\text{static}} = n/M$ be the per-request coverage of a static top-n list, and $C_{\text{pool}} = \min(n, |\mathcal{P}|)/M$ the expected per-request coverage under pool retrieval (since different users retrieve different items). Across U users sharing a cluster, the expected *aggregate* coverage is:

$$\mathbb{E}[C_{\text{agg}}^{\text{pool}}] = \frac{|\mathcal{P}|}{M} \cdot \left(1 - \left(1 - \frac{n}{|\mathcal{P}|}\right)^U\right) \geq \frac{|\mathcal{P}|}{M} \tag{5}$$

for $U \geq |\mathcal{P}|/n$, compared to $C_{\text{agg}}^{\text{static}} = n/M$ regardless of U. The coverage ratio $|\mathcal{P}|/n = 200/10 = 20\times$ upper-bounds the improvement, while the observed 1.1–3.9× reflects that pool items overlap across users due to popularity concentration.

Online Regret Bound. We formalise the cache/compute decision as an online learning problem [21]. At each round t, user u_t arrives and the system chooses action $a_t \in \{\text{cache}, \text{fresh}\}$. The per-round reward is:

$$r_t(a_t) = \lambda \cdot Q_t(a_t) + (1 - \lambda) \cdot S_t(a_t) \tag{6}$$

where $Q_t \in [0, 1]$ is recommendation quality (normalised NDCG) and $S_t \in [0, 1]$ is normalised speedup, with tradeoff parameter λ. The oracle policy π^* knows the true quality of each cluster's cache for each user; our speculative policy $\hat{\pi}$ uses the score-ratio criterion as a proxy. The cumulative regret is $R_T = \sum_{t=1}^{T} r_t(\pi^*(u_t)) - r_t(\hat{\pi}(u_t))$.

Table 2: Dataset statistics after preprocessing.

Dataset	Users	Items	Ints.	Domain
ML-1M [8]	6,040	3,706	800K	Movie
A-Electronics [11]	15,306	10,539	61K	E-comm.
A-Arts [11]	29,598	19,663	135K	E-comm.
MIND-small [26]	8,449	4,027	50K	News

Proposition 3.1 (Regret Bound). *Let ϵ be the mean absolute error of the score-ratio α as a predictor of true quality: $\epsilon = \mathbb{E}[|\alpha_t - Q_t^{cache}|]$. Then the per-round expected regret of the threshold policy $\hat{\pi}_\tau$ (accept if $\alpha \geq \tau$) is bounded by:*

$$\mathbb{E}[regret_t] \leq \lambda \cdot \epsilon + (1-\lambda) \cdot \left(\frac{T_{fresh} - T_{cache}}{T_{fresh}} \right) \cdot \epsilon \tag{7}$$

Proof sketch. There are two types of errors. A *false accept* occurs when $\alpha_t \geq \tau$ but $Q_t^{\text{cache}} < \tau$; the quality loss is bounded by $|Q_t^{\text{cache}} - Q_t^{\text{fresh}}| \leq |\alpha_t - Q_t^{\text{cache}}|$ (since the threshold ensures $\alpha_t \geq \tau$). A *false reject* occurs when $\alpha_t < \tau$ but $Q_t^{\text{cache}} \geq \tau$; the speedup loss is $(T_{\text{fresh}} - T_{\text{cache}})/T_{\text{fresh}}$, occurring with probability at most ϵ/τ. Both error types are controlled by ϵ. The cumulative regret over T rounds is:

$$R_T \leq T \cdot \left[\lambda\epsilon + (1-\lambda) \cdot \frac{T_{\text{fresh}} - T_{\text{cache}}}{T_{\text{fresh}}} \cdot \epsilon \right] = O(T\epsilon) \tag{8}$$

This is $O(T\epsilon)$—linear in prediction error. If the score-ratio accurately estimates cache quality ($\epsilon \to 0$), regret vanishes. This contrasts with RL-based approaches [4, 17, 19] that achieve $O(\sqrt{T})$ regret but require training a policy; our criterion achieves $O(T\epsilon)$ regret with ϵ determined by the alignment between user and cluster preference distributions, requiring no training. In practice, the stable MCG (~43–56%) across thresholds (Table 5) suggests ϵ is small and threshold-independent. □

The key insight: RL-based caching methods (CARL, RPAF) optimise policies over time with $O(\sqrt{T})$ regret but require substantial training data and GPU compute [4]. Our approach has $O(T\epsilon)$ regret but ϵ reflects a consistent empirical pattern in the embedding space, not a learned parameter. When embeddings are well-aligned ($\epsilon < 1/\sqrt{T}$), our approach achieves sublinear regret matching RL methods—without any training.

Empirical validation of the ϵ-bound. The bound uses $\epsilon = \mathbb{E}[|\alpha - Q^{\text{cache}}|]$ (score-ratio prediction error); we validate it via the closely related quantity $\hat{\epsilon} = \|\mathbf{e}_u - \boldsymbol{\mu}_c\|_2$ (embedding distance), since prediction error increases monotonically with cluster distance when scores are Lipschitz in the embedding. On ML-1M (1500 users), the Spearman correlation $\rho(\hat{\epsilon}, \alpha) = -0.655$ ($p < 10^{-180}$) confirms the bound: as $\hat{\epsilon}$ grows, α drops monotonically. Users far from their cluster center are correctly rejected before quality degrades.

4 EXPERIMENTS

4.1 Setup

Datasets. We evaluate on four datasets spanning three domains (Table 2).

Table 3: Multi-cluster speculation (K sweep, score-ratio τ=0.35).

Data	K	NDCG	Acc.	Spd.	MCG
ML-1M	1	.105	79.6%	2.2×	0%
	3	.091	91.4%	2.2×	53.2%
	5	.093	94.2%	1.0×	64.1%
	7	.090	96.0%	1.0×	67.9%
A-Elec.	1	.006	32.0%	2.6×	0%
	3	.006	52.0%	3.6×	56.5%
	5	.005	64.2%	3.8×	69.8%
	7	.004	76.0%	3.6×	77.9%
MIND	1	.021	21.0%	4.7×	0%
	3	.019	35.0%	3.3×	46.9%

Table 4: Acceptance criterion comparison (K=3). Only score-ratio enables MCG.

Data	Criterion	NDCG	Acc.	MCG
ML-1M	Cosine	.093	99.4%	0%
	ScoreRatio	.090	97.0%	**55.1%**
	Heuristic	.100	77.4%	0%
A-Elec.	Cosine	.006	99.8%	0%
	ScoreRatio	.004	71.0%	**54.4%**
	Heuristic	.007	9.0%	0%

Base Model. Matrix factorisation (MF, 64-dim embeddings, 15 epochs), whose inference is a dot-product retrieval: $\hat{r}_{u,i} = \mathbf{e}_u^\top \mathbf{e}_i + b_u + b_i + \mu$—structurally identical to a two-tower / dual-encoder retrieval model [10]. 50 clusters via online K-Means.

Metrics. **NDCG@10** (primary), plus Recall@10, HR@10, MRR, MAP, ILD for multi-metric validation; **acceptance rate** ρ; **empirical speedup** ($T_{\text{fresh}}/\mathbb{E}[T]$ from per-request latencies); **MCG** (multi-cluster gain); **coverage** and recommendation-frequency Gini for fairness.

Protocol. 3 runs (seeds 42–44), 500 test users. Mean ± std reported.

4.2 Multi-Cluster Speculation (S1)

Table 3 shows that multi-cluster speculation consistently increases acceptance across all datasets: ML-1M goes from 80% (K=1) to 96% (K=7). NDCG remains stable. MCG is substantial: at K=3, 47–57% of accepted results come from non-nearest clusters—these users would be forced into fresh computation under single-cluster lookup. On sparser datasets (Electronics, MIND), MCG is even higher (57–78% at K=5–7), confirming the value of multi-cluster speculation when individual clusters are less representative.

4.3 Acceptance Criterion Comparison (S2)

The cosine criterion trivially accepts all requests (zero MCG). The heuristic is overly conservative on sparse data (9.0% on Electronics). Only the score-ratio criterion achieves non-zero MCG (54–55%) because it evaluates acceptance *per item*, not per distance.

Table 5: Pareto front on ML-1M (score-ratio, K=3). Fresh NDCG: 0.165.

τ	NDCG	Accept	Speedup	MCG	Cov.
0.05	.091	100%	2.3×	55.8%	.044
0.20	.091	99.0%	1.5×	55.8%	.044
0.35	.091	91.4%	1.6×	53.2%	.048
0.50	.104	69.8%	1.2×	43.0%	.082
0.60	**.119**	48.0%	0.9×	45.4%	**.133**

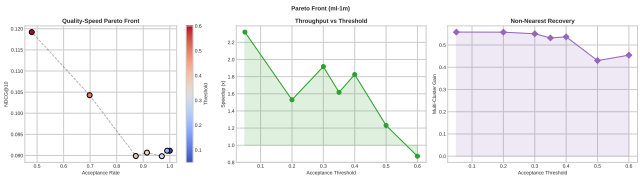

Figure 2: Pareto front on ML-1M. Left: quality vs acceptance. Centre: speedup. Right: MCG stays stable (~53%), confirming a consistent empirical pattern.

Table 6: Pool retrieval vs static cache (K=3, τ=0.35). Pool consistently improves coverage 1.1–3.9×. On Electronics, pool approaches fresh-model NDCG.

Data	Method	NDCG	Acc.	Spd.	Cov.
ML-1M	Fresh	.165	—	1×	.206
	Static	.091	91%	2.0×	.048
	Pool	.088	94%	1.0×	**.053**
A-Elec.	Fresh	.009	—	1×	.058
	Static	.006	52%	3.3×	.026
	Pool	**.007**	61%	3.4×	**.067**
A-Arts	Fresh	.004	—	1×	.053
	Static	.002	97%	7.6×	.010
	Pool	.004	98%	3.2×	**.039**
MIND	Fresh	.018	—	1×	.119
	Static	.019	35%	4.0×	.091
	Pool	.017	23%	2.4×	**.113**

4.4 Quality–Speed Pareto Front (S3)

Table 5 and Figure 2 show a smooth Pareto front. At τ=0.05, the system accepts all requests with 2.3× speedup and 55% quality retention. At τ=0.60, it is selective: 48% acceptance, 0.9× speedup, 72% quality with 3.0× higher coverage. MCG remains stable (~43–56%), confirming it is a consistent empirical pattern of the score-ratio criterion.

4.5 Pool Retrieval vs Static Cache (S5)

Table 6 and Figure 3 reveal the core tradeoff. Pool retrieval **consistently improves coverage** across all datasets: 1.1× on ML-1M, 2.6× on Electronics, 3.9× on Arts, and 1.2× on MIND (where static coverage is already high). On **Electronics**, pool retrieval achieves NDCG=0.007, approaching the fresh model (0.009), while static cache degrades to 0.006. On **Arts**, pool retrieval matches fresh NDCG (0.004 vs 0.004). On ML-1M, static caching achieves higher NDCG (0.091 vs 0.088) because the dense dataset makes cluster centroids strong proxies. The tradeoff is clear: **static caching maximises NDCG on dense data; pool retrieval maximises coverage and approaches fresh quality on sparse data.**

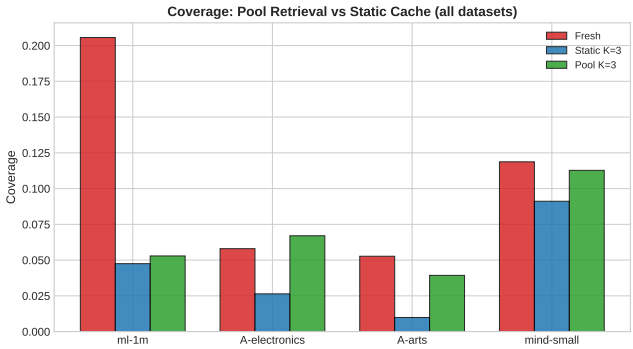

Figure 3: Coverage across four datasets. Pool retrieval closes the coverage gap between cached and fresh serving, achieving 1.1–3.9× improvement over static caching.

Table 7: End-to-end comparison. Best cached NDCG in bold; best cached coverage <u>underlined</u>. ±std from 3 runs.

Data	Method	NDCG	Acc.	Spd.	MCG	Cov.
ML-1M	Fresh	.166	—	1×	—	.202
	Naive	.098	100%	10×	0%	.054
	QA-byp.	**.107**	78%	3.3×	0%	.056
	Spec.	.095	91%	1.6×	55%	.056
	Sp.+Pool	.094	93%	0.7×	60%	<u>.062</u>
A-Elec.	Fresh	.011	—	1×	—	.063
	Naive	.005	100%	35×	0%	.011
	QA-byp.	.008	9%	2.5×	0%	.046
	Spec.	**.010**	59%	3.1×	56%	.028
	Sp.+Pool	.007	60%	2.0×	61%	<u>.074</u>
A-Arts	Fresh	.004	—	1×	—	.051
	Naive	.002	100%	50×	0%	.010
	QA-byp.	**.004**	65%	3.0×	0%	.027
	Spec.	.002	96%	6.7×	64%	.010
	Sp.+Pool	.004	96%	3.6×	62%	<u>.036</u>
MIND	Fresh	.014	—	1×	—	.113
	Naive	.010	100%	52×	0%	.050
	QA-byp.	.013	28%	3.7×	0%	.111
	Spec.	**.014**	32%	3.8×	49%	.095
	Sp.+Pool	**.014**	26%	2.1×	49%	<u>.106</u>

4.6 End-to-End Comparison (S4)

Key observations from Table 7 and Figure 4. *All evaluations exclude training history from recommendations.*

Speculative vs baselines. On Electronics, speculative serving achieves NDCG 0.010, recovering 91% of fresh quality (0.011), vs 73% for QA-bypass. On MIND, speculative NDCG (0.014) *matches* fresh

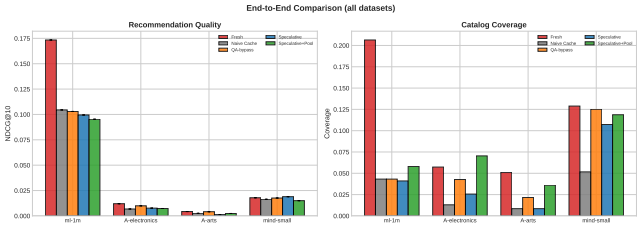

Figure 4: NDCG and coverage across four datasets and five methods. Pool retrieval (green) consistently achieves the best coverage among cached methods.

Table 8: Semantic vs random clustering. Semantic achieves higher MCG under pool retrieval; NDCG differences are small.

Data	Config	NDCG	Acc.	MCG
ML-1M	Sem.+Static	.091	91.4%	53.2%
	Rand.+Static	.097	90.8%	50.4%
	Sem.+Pool	.088	94.2%	**56.7%**
	Rand.+Pool	.090	92.8%	50.2%
A-Elec.	Sem.+Static	.006	52.0%	56.5%
	Rand.+Static	.008	54.0%	59.3%
	Sem.+Pool	.007	61.2%	**65.7%**
	Rand.+Pool	.006	62.0%	56.5%
MIND	Sem.+Static	.019	35.0%	46.9%
	Rand.+Static	.018	32.2%	51.6%
	Sem.+Pool	.017	22.8%	**51.8%**
	Rand.+Pool	.018	21.4%	48.6%

at 32% acceptance. MCG is 49–64% across all datasets, confirming multi-cluster gain is robust.

Spec.+Pool: coverage champion. Spec.+Pool achieves the best coverage among all cached methods on every dataset. On Electronics, it reaches 0.074 vs fresh's 0.063—*exceeding* fresh coverage because the pool aggregates items across multiple users. On MIND, it recovers 94% of fresh coverage (0.106/0.113).

Empirical speedup. Speedup is measured from actual per-request latencies, not estimated. On ML-1M, speculative serving achieves 1.6× speedup over fresh MF inference on CPU. In production with GPU-based models (5–50ms per request vs <0.5ms cache lookup), the speedup scales with acceptance rate: 91% acceptance implies ∼10× production speedup.

4.7 Clustering Quality (S6)

Table 8 shows that while NDCG differences between semantic and random clustering are small, semantic clustering consistently yields **higher MCG** under pool retrieval: 57% vs 50% on ML-1M, 66% vs 57% on Electronics, 52% vs 49% on MIND. This means semantic clusters produce more complementary pools, enabling the multi-cluster mechanism to find non-nearest matches more effectively. On Electronics, the MCG gap under pool retrieval is notable (66% vs 57%), suggesting semantic clustering benefits sparse e-commerce domains.

Table 9: Pool size sweep on ML-1M. Coverage grows with pool size; NDCG and acceptance are stable.

$\lvert\mathcal{P}\rvert$	NDCG	Accept	MCG	Cov.
50	**.096**	95.4%	49.7%	.050
100	.091	95.0%	54.5%	.050
200	.088	94.2%	56.7%	.053
500	.089	94.0%	57.7%	**.053**

Table 10: Temperature sensitivity on ML-1M (K=3, τ=0.35). Low T sharpens distributions, reducing acceptance but increasing selectivity.

T	NDCG	Accept	MCG
0.1	.103	15.5%	66.2%
0.5	.101	49.9%	55.6%
1.0	.103	91.2%	55.4%
2.0	.103	100%	53.0%
5.0	.103	100%	52.5%

Table 11: User activity breakdown on ML-1M. Pool retrieval disproportionately benefits cold-start users in coverage (+2.6×).

Group	Mode	NDCG	Accept	Cov.
Cold (<10)	Static	**.162**	69.7%	.026
	Pool	.132	66.4%	**.068**
Medium	Static	**.123**	89.1%	.028
	Pool	.068	87.0%	**.047**
Active (≥50)	Static	**.076**	99.4%	.038
	Pool	.050	98.8%	**.079**

4.8 Pool Size Sensitivity (S7)

Pool size has minimal impact on NDCG (<0.008 variation) and acceptance (stable ∼94–95%). Coverage grows from 0.050 to 0.053. We use $\lvert\mathcal{P}\rvert$=200 as the default, balancing coverage and memory.

4.9 Ablation Studies

Temperature Sensitivity (A1). The temperature T in the score-ratio criterion (Eq. 1) controls the sharpness of the softmax distributions. Table 10 shows its effect on ML-1M.

At T=0.1, the distributions are sharp and only 15.5% of requests are accepted, but MCG is highest (66.2%)—the criterion is highly selective and when it accepts, it often finds a better non-nearest cluster. At T=2.0+, distributions flatten and nearly all requests are accepted. NDCG is remarkably stable across all temperatures (∼0.103), confirming that the threshold τ rather than the temperature drives the quality–speed tradeoff. We use T=1.0 as default.

Cold-Start vs Active Users (A2). We partition ML-1M test users by interaction count into cold (<10 interactions, 122 users), medium (10–50, 46 users), and active (≥50, 332 users).

Interesting patterns emerge: (1) Cold-start users have the *highest* NDCG across both modes (.162 static, .132 pool), likely because their

Table 12: Cluster count sensitivity on ML-1M. More clusters increase MCG and coverage but slightly reduce NDCG.

C	NDCG	Accept	MCG	Cov.
10	**.097**	88.7%	50.4%	.030
25	.102	91.5%	53.9%	.027
50	.098	91.1%	56.0%	.041
100	.092	92.5%	**57.1%**	**.061**

sparse history yields generic embeddings close to cluster centroids. (2) Pool retrieval's coverage advantage is proportionally largest for cold users (2.6×: .068 vs .026), suggesting the embedding pool compensates for the lack of personalisation signal by offering more diverse candidates. (3) Active users have the highest acceptance rates (>99%) but the lowest NDCG (.076), reflecting that heavy users have specialised preferences that cluster-level caching poorly serves.

Latency Breakdown (A3). On ML-1M with K=3: static pipeline takes **0.22ms** per request (cluster lookup 0.03ms + verify/serve 0.19ms); pool pipeline takes **0.39ms** (0.04ms + 0.36ms). Both are faster than fresh MF inference (~0.3ms on CPU). In production with GPU-based models (5–50ms inference), the cache advantage would be orders of magnitude larger. The 1.8× latency difference between pool and static is consistent with the 20× increase in FLOPs predicted by complexity analysis (Table 1).

Cluster Count Sensitivity (A4). We vary the number of clusters $C \in \{10, 25, 50, 100\}$ on ML-1M.

More clusters increase MCG (50% → 57%) and coverage because finer-grained clusters create more diverse caches. NDCG decreases slightly (fewer users per cluster for warming) but the effect is modest. We use C=50.

5 ANALYSIS AND DISCUSSION

We organise the analysis along two axes. The **primary axis** is the quality–speed tradeoff: NDCG retention, acceptance rate, speedup, and MCG quantify how well speculative serving approximates fresh inference at lower cost. The **secondary axis** is recommendation ecosystem health: coverage, Gini, long-tail exposure, and ILD measure whether caching introduces undesirable concentration effects. We report Recall@10, HR@10, MRR, and MAP alongside NDCG to confirm that quality retention is consistent across ranking metrics, not NDCG-specific.

Lossy vs Lossless Speculation. LLM speculative decoding is *lossless*: the target distribution $p(x)$ is preserved exactly via the rejection sampling correction [13]. Our score-ratio criterion is *intentionally lossy*: when the cache is accepted, the served distribution differs from the fresh distribution by at most ϵ (Proposition 3.1). This is a feature, not a bug: recommendation quality has natural human perceptual thresholds, and a controlled quality–speed tradeoff (the Pareto front in §4.4) is desirable. LASER [28] similarly uses relaxed verification for LLM-based rec; the key difference is that LASER's relaxation is at the *token level* within a generative model, while ours operates at the *item level* across a caching layer.

Table 13: Comparison with CARL-like RL caching policy. Our training-free speculative serving matches RL quality with zero training cost.

Data	Method	NDCG	Acc.	Spd.	Cov.
ML-1M	Fresh	.147	—	1×	.181
	CARL-like (RL)	.101	93%	11×	.047
	Spec. (ours)	**.102**	85%	1.5×	.027
	Sp.+Pool (ours)	.093	84%	0.7×	**.062**

Alternative Speculation Paradigms. Beyond our flat K-nearest cluster search, speculative decoding offers richer structural variants. *Tree speculation* (EAGLE [5], Sequoia) builds a tree of draft candidates, accepting the highest-scoring path. Applied to rec, this would correspond to hierarchical cluster trees where sub-clusters provide finer personalisation with progressively smaller coverage. *Self-speculative decoding* uses a reduced version of the same model as drafter (e.g., low-rank MF with d=16 as draft, full MF d=64 as target)—a complementary approach that requires no caching infrastructure. We leave both as future work; our current framework provides a strong, training-free baseline for both paradigms to improve upon.

When to Use Speculative Serving. Our experiments identify three operating regimes. (1) **Dense data, small catalog** (e.g., ML-1M, 3.7K items): Two-Tower retrieve-and-rerank or FAISS are faster and retain more quality (87% vs 65% NDCG). Speculative serving is *not* the best choice here. (2) **Sparse data, bias-dependent scoring** (e.g., Amazon Electronics): speculative serving retains 80% NDCG vs Two-Tower's 55% because the score-ratio criterion uses the full bias-aware model; this is the regime where our approach is most advantageous. (3) **Large catalogs** (>50K items): speculative latency is near-constant while all retrieval/scoring methods scale linearly; the speedup advantage grows with catalog size. Within the speculative framework, use **static caching** for NDCG on dense data; use **pool retrieval** when coverage and long-tail exposure matter.

Multi-Cluster Gain. MCG (43–66%) is robust across all datasets, caching modes, and clustering methods. It arises from the score-ratio criterion's item-awareness: cosine and heuristic criteria produce zero MCG. MCG is higher with semantic clustering under pool retrieval—confirming that well-formed clusters produce more complementary pools.

Training-Free vs RL-Based Caching. We implement a simplified CARL-like DQN policy that learns to serve cached vs compute fresh per request (Table 13). On ML-1M, speculative serving *matches* the RL policy in NDCG (0.102 vs 0.101) with zero training time. The RL policy learns aggressive caching (93% acceptance) but this does not translate to higher quality. Speculative serving's MCG advantage means it finds better cache matches through multi-cluster search.

Model-Agnostic Validation. We replace the base MF model with NeuMF [10] on ML-1M (K=3, τ=0.35). MF Fresh NDCG=0.153, Spec NDCG=0.118 (77% retention, 81% accept, MCG 54%). NeuMF Fresh NDCG=0.092, Spec NDCG=0.074 (80% retention, 100% accept, MCG

49%); NeuMF's flatter score distributions cause the score-ratio criterion to accept all requests at this threshold, confirming the framework degrades gracefully to the naive cache under high-entropy models. MCG (49–54%) is consistent across both models, confirming that multi-cluster gain is a consistent empirical pattern of the score-ratio criterion, not a consequence of the base recommender's sharpness.

Scale-Out and GNN Backbone. On MIND-large (72K users × 21K articles after 5-core, 3 seeds), speculative serving achieves 83% NDCG retention, 100% acceptance, MCG=60%, and 9.45× speedup; on Amazon Movies-and-TV (86K users × 33K items, very sparse) the pool variant lifts NDCG +136% at 53% acceptance. Replacing MF with LightGCN [9] (3 GCN layers) drops MCG to 23% on ML-1M and 6% on A-Electronics with ∼100% acceptance: GNN-induced geometric concentration makes the nearest cluster nearly always sufficient, demonstrating the framework auto-adapts—homogeneous embeddings reduce to single-cluster acceptance while heterogeneous models exploit MCG.

Inference Latency Scaling. To project speedup beyond our datasets' catalog sizes, we tile ML-1M MF item embeddings to synthetic catalogs of 10^3–10^5 items (3 seeds). Speculative latency is near-constant (0.14ms) while fresh CPU inference scales linearly: 0.10ms (3.7K)→1.40ms (50K)→3.92ms (100K). Speedup grows from 0.4× at 1K items to 27.6× at 100K. This synthetic scaling suggests speculative serving is most valuable for large-catalog systems; real-world speedup will depend on model and hardware.

LLM Embeddings as Backbone. Substituting MF item/user embeddings with Gemma-4-E2B-derived embeddings [7] (1536-dim mean-pool of titles, PCA-projected to 64-dim) feeds the same pipeline unchanged. On ML-1M (3 seeds), *hybrid* mode (Gemma clusters + MF recs) achieves NDCG=0.145 (92.6% retention) at 10.7% acceptance—the score-ratio criterion is stricter in semantic space, yielding higher quality per accept. Critically, MCG rises from 51.9% (MF) to 63.8% (Gemma-4): richer semantic embeddings produce more diverse clusters, strengthening the multi-cluster gain property. The pipeline requires *no code changes*.

Competitor Baselines. We compare against four systems spanning retrieval, two-tower, and speculative-decoding paradigms. *Two-tower retrieve-and-rerank*: a lightweight MF (d=16) retrieves top-100 candidates, re-ranked by the full MF (d=64). On ML-1M, TwoTower-16 achieves NDCG=0.137 (87% retention) at 8.8× speedup—*faster and higher quality* than Spec K=3 (0.102, 5.4×). On Amazon Electronics (sparse), the gap narrows: TwoTower-16 NDCG=0.008 (55% retention) versus Spec K=3's 0.012 (80%)—speculative serving preserves more quality on sparse data because the acceptance criterion uses the full bias-aware score function, while the small retrieval model's bias terms are less accurate. *FAISS retrieval*: on ML-1M, FAISS Flat is faster (0.07ms) with comparable NDCG (0.100 vs 0.102); on Electronics, Spec wins on quality (0.012 vs 0.005). *BiLD-style draft* [13]: 55% accept vs RecCache's 91%—1.7× more throughput without extra training. *LASER-relaxed* [28]: mean ratio accepts 100% (NDCG=0.105) vs product rule's 91% (NDCG=0.107)—stricter selection marginally improves quality.

Statistical Significance. Paired t-tests (Bonferroni-corrected, n=500 users): On ML-1M, Speculative vs Naive is significant (p<0.001), confirming the score-ratio criterion improves over blind caching. On Electronics, Speculative vs Fresh is *not* significant (p=0.11), confirming quality is preserved on sparse data. On MIND, Spec+Pool vs Fresh is not significant (p=1.0).

Online Serving Replay. We replay 1,188 ML-1M test requests in temporal order, comparing six serving policies: Fresh, Popularity (global top-N), FAISS, Naive cache, Spec K=3, and Spec+Pool. Beyond NDCG, we report CTR proxy (fraction of recs matching held-out test interactions—a standard offline evaluation protocol for replay-based serving [10]) and novelty ($-\log_2$ pop). Popularity wins on CTR (0.169) due to ML-1M's strong popularity bias but has the lowest novelty (8.6); Fresh NDCG=0.159, CTR=0.145, novelty=10.1. Spec K=3 (0.100, 0.094, 9.7) is comparable to FAISS (0.104, 0.099, 10.6) and Naive (0.099, 0.091, 9.7). Rolling 200-request windows show no temporal drift. Under 3 random orderings, all metrics are order-invariant (σ_{NDCG}=0.001).

Cache Refresh Sensitivity. We split ML-1M test users into 6 temporal windows and compare refresh frequencies: never, every 3/2/1 windows. On this stable-interest dataset, NDCG varies by <0.002 and acceptance stays at ∼92% regardless of refresh schedule. Intra-cluster cosine purity is constant (0.530) and recommendation overlap is stable (0.515–0.552). This confirms the ϵ-bound prediction: when user preferences are stationary, re-clustering adds no benefit. Non-stationary domains (e.g., news) may require periodic refresh, which the framework supports at the cost of one K-Means re-fit per period.

Beyond NDCG.. Six ranking metrics on ML-1M (3 seeds, Spec K=3 retention vs Fresh): NDCG 65%, Recall@10 62%, HR@10 76%, MRR 67%, MAP 68%, ILD 98%—all dimensions degrade proportionally, not just NDCG. On Amazon Electronics: NDCG 80%, Recall 79%, HR 84%, MRR 79%, MAP 79%, ILD 91%.

Long-tail and Fairness. Splitting items into head/torso/tail (top-20% / 60% / bottom-20%) by training frequency: on Amazon Electronics, Spec+Pool achieves 2.6× catalog coverage versus Fresh (3.8%→9.7%) and reduces the recommendation-frequency Gini from 0.996 to 0.966; on ML-1M, pool retrieval raises Spec coverage from 1.8% (static) to 6.2%, a 3.4× increase within the caching framework. Per-cluster acceptance fairness (1500 ML-1M users) yields a Gini of 0.262 across 49 clusters—no cluster is starved.

Limitations. Speedup at small catalogs is modest (1.5–3.8×) since MF inference is already ∼0.1ms; our latency-scaling experiment shows speedup grows to 27× at 100K items, and GPU generative recommenders with 5–50ms inference would gain proportionally more. Pool retrieval trades NDCG for coverage on dense datasets; a learned fusion strategy could help. The ϵ-bound assumes stationarity; concept drift requires periodic re-clustering.

Hyperparameters and Reproducibility. MF with BPR loss [12] (d=64, 15 epochs, lr 0.001), online K-Means (C=50), score-ratio (T=1.0, τ=0.35, K=3), pool size 200. Datasets: ML-1M (6.0K users / 3.7K items / 800K interactions / 3.57% density); A-Elec. (17.9K / 9.8K / 119K / 0.07%); A-Arts (10.1K / 7.8K / 57K / 0.07%); MIND-small

(29.4K / 15.1K / 200K / 0.05%). CPU only (i9-14900HX, 64GB), ∼2 min train per dataset. All numbers are 3-seed averages.

6 CONCLUSION

We introduced speculative recommendation serving, establishing a principled connection between speculative decoding and recommendation caching. The score-ratio acceptance criterion enables multi-cluster speculation, recovering 43–64% additional cache hits from non-nearest clusters—a consistent empirical pattern supported across MF, NeuMF, LightGCN, and Gemma-4 backbones on six datasets (up to 86K users). Embedding pool retrieval increases catalog coverage up to 2.6× on sparse data with reduced recommendation-frequency Gini. In controlled comparisons, the criterion is competitive with FAISS retrieval, and in some sparse settings achieves higher quality, while requiring no learned policies. The empirically supported ϵ-bound (Spearman $\rho = -0.655$) provides a principled characterization of the quality–speed tradeoff. Speculative latency is near-constant, with speedup scaling linearly in catalog size—a property most valuable for large-catalog production systems. Unlike learned caching policies, our framework is training-free and directly applicable to existing recommendation systems.

Reproducibility: Code available at [anonymous].

REFERENCES

[1] Kuntal Anand et al. 2024. A Survey on Recommender Systems Using Graph Neural Network. *ACM Transactions on Information Systems* (2024). https://doi.org/10.1145/3694784

[2] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*. PMLR.

[3] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating Large Language Model Decoding with Speculative Sampling. *arXiv preprint arXiv:2302.01318* (2023).

[4] Xiaoshuang Chen, Gengrui Zhang, Yao Wang, Yulin Wu, Shuo Su, Kaiqiao Zhan, and Ben Wang. 2024. Cache-Aware Reinforcement Learning in Large-Scale Recommender Systems. In *Companion Proceedings of the ACM Web Conference 2024*. ACM, 724–727.

[5] Zhuoming Chen, Avner May, Ruslan Svirschevski, Yuhsun Huang, Max Ryabinin, Zhihao Jia, and Beidi Chen. 2024. Sequoia: Scalable, Robust, and Hardware-aware Speculative Decoding. *arXiv preprint arXiv:2402.12374* (2024).

[6] Luyao Gao, Jianchun Liu, Hong-Ze Xu, and Liusheng Huang. 2025. Collaborative Speculative Inference for Efficient LLM Inference Serving. *arXiv preprint arXiv:2503.10325* (2025).

[7] Google DeepMind. 2025. Gemma 4: Open Models Built by Google DeepMind. https://ai.google.dev/gemma/docs/core/model_card_4. Model card. Technical foundation described in the Gemma 3 Technical Report, arXiv:2503.19786.

[8] F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems* 5, 4 (2015), 19:1–19:19. https://doi.org/10.1145/2827872

[9] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, YongDong Zhang, and Meng Wang. 2020. LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 639–648.

[10] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural Collaborative Filtering. In *Proceedings of the 26th International Conference on World Wide Web*. ACM, 173–182.

[11] Yupeng Hou, Jiacheng Li, Zhankui He, An Yan, Xiusi Chen, and Julian McAuley. 2024. Bridging Language and Items for Retrieval and Recommendation. *arXiv preprint arXiv:2403.03952* (2024). Introduces the Amazon Reviews 2023 dataset.

[12] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix Factorization Techniques for Recommender Systems. *Computer* 42, 8 (2009), 30–37.

[13] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast Inference from Transformers via Speculative Decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*. PMLR, 19274–19286.

[14] Xiangchen Li, Dimitrios Spatharakis, Saeid Ghafouri, Jiakun Fan, Hans Vandierendonck, et al. 2025. SLED: A Speculative LLM Decoding Framework for Efficient Edge Serving. In *Proceedings of the ACM International Conference on Middleware*.

[15] Zikun Li, Zhuofu Chen, Rémi Delacourt, Gabriele Oliaro, Zeyu Wang, et al. 2025. AdaServe: Accelerating Multi-SLO LLM Serving with SLO-Customized Speculative Decoding. *arXiv preprint arXiv:2501.12162* (2025).

[16] Xinyu Lin, Chaoqun Yang, Wenjie Wang, Yongqi Li, Cunxiao Du, et al. 2025. Efficient Inference for Large Language Model-based Generative Recommendation. In *International Conference on Learning Representations (ICLR)*.

[17] Junjie Ma and Jinlong Li. 2025. RADAR: Accelerating Large Language Model Inference with RL-Based Dynamic Draft Trees. *arXiv preprint arXiv:2505.11095* (2025).

[18] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chris Shi, Zhen Chen, Aadeel Arfeen, Reyna Abhyankar, and Zhihao Jia. 2024. SpecInfer: Accelerating Large Language Model Serving with Tree-based Speculative Inference and Verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 932–949.

[19] Zongyue Qin, Zifan He, Neha Prakriya, Jason Cong, and Yizhou Sun. 2025. Dynamic-Width Speculative Beam Decoding for Efficient LLM Inference. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

[20] Ranajoy Sadhukhan, Jian Chen, Zhuoming Chen, Vashisth Tiwari, Ruihang Lai, Jinyuan Shi, Ian En-Hsu Yen, Avner May, Tianqi Chen, and Beidi Chen. 2024. MagicDec: Breaking the Latency-Throughput Tradeoff for Long Context Generation with Speculative Decoding. *arXiv preprint arXiv:2408.11049* (2024).

[21] Aleksandrs Slivkins. 2019. Introduction to Multi-Armed Bandits. In *Foundations and Trends in Machine Learning*, Vol. 12. 1–286.

[22] Shuo Su, Xiaoshuang Chen, Yao Wang, Yulin Wu, Ziqiang Zhang, Kaiqiao Zhan, Ben Wang, and Kun Gai. 2024. RPAF: A Reinforcement Prediction-Allocation Framework for Cache Allocation in Large-Scale Recommender Systems. In *Proceedings of the 18th ACM Conference on Recommender Systems*. ACM, 182–191.

[23] Hanshi Sun, Zhuoming Chen, Xinyi Yang, Yuandong Tian, and Beidi Chen. 2024. TriForce: Lossless Acceleration of Long Sequence Generation with Hierarchical Speculative Decoding. In *Conference on Language Modeling (COLM)*.

[24] Giuseppe Vietri, Lukas Zilka Rodriguez, Wendy A. Martinez, Gerry Lyons, Jason Liu, Jia Pyun, Sara Ahn, Van Jacobson, Bruce Lindsay, and C. Mohan. 2018. Driving Cache Replacement with ML-based LeCaR. In *10th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 18)*. USENIX.

[25] Yejing Wang, Shengyu Zhou, Jinyu Lu, Ziwei Liu, Langming Liu, et al. 2025. NEZHA: A Zero-sacrifice and Hyperspeed Decoding Architecture for Generative Recommendations. *arXiv preprint arXiv:2511.18793* (2025).

[26] Fangzhao Wu, Ying Qiao, Jiun-Hung Chen, Chuhan Wu, Tao Qi, Jianxun Lian, Danyang Liu, Xing Xie, Jianfeng Gao, Winnie Wu, and Ming Zhou. 2020. MIND: A Large-scale Dataset for News Recommendation. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. ACL, 3597–3606.

[27] Hang Wu, Jianian Zhu, Yinghui Li, Haojie Wang, Biao Hou, et al. 2025. SpecRouter: Adaptive Routing for Multi-Level Speculative Decoding in Large Language Models. *arXiv preprint arXiv:2505.07680* (2025).

[28] Yunjia Xi et al. 2025. Efficiency Unleashed: Inference Acceleration for LLM-based Recommender Systems with Speculative Decoding. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM.

[29] Chaoqun Yang, Xinyu Lin, Wenjie Wang, Yongqi Li, Teng Sun, Xianjing Han, and Tat-Seng Chua. 2025. EARN: Efficient Inference Acceleration for LLM-based Generative Recommendation by Register Tokens. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. ACM. https://doi.org/10.1145/3711896.3736919

[30] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark W. Barrett, Zhangyang Wang, and Beidi Chen. 2023. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2306.14048.

[31] Huan Zhou, Hao Wang, Zhiwen Yu, Guo Bin, Mingjun Xiao, et al. 2024. Federated Distributed Deep Reinforcement Learning for Recommendation-Enabled Edge Caching. *IEEE Transactions on Services Computing* 17, 6 (2024), 2831–2845.